

AI driven development in one week

A 36 hour 30 minute practical study program for developers and Web3 developers • 16 September 2026 edition

Clément Lesaëge + Astra

Use capable agents to take a development task from an ambiguous request to a verified change. You will learn to specify outcomes, give agents useful tools and feedback, delegate selectively, and turn a successful process into a repeatable routine. No previous agent knowledge is assumed; ordinary development skills are required. Start as a developer managing an intern: give very explicit instructions and check everything the agent does. As your proficiency grows, evolve toward a department manager's role, coordinating multiple AI workers around outcomes, dependencies and acceptance criteria.

Approach the week as a rediscovery exercise. An experience with an older model, prompt or IDE is evidence about that setup at that time. Current agents can sustain longer work and need different scaffolding. Test your old assumptions—including elaborate prompts and forced decomposition—against current tools.

Day	Focus	Time
1	Rediscovery, models, plans and harness setup	7h30
2	Prompting, grilling and independent verification	7h
3	Skills, context, goals and bounded loops	7h
4	Graphs, Ultra and managing parallel work	8h
5	Routines, a small wrapper and a capstone	7h

How to follow the program

Follow each day's schedule and open the numbered resource cards when directed. Each resource code opens its source; for a block with several sources, it opens the first one and the card links the rest. Cards give the reading or viewing scope, an approximate duration and an expert-only skip condition. The week has a 36h30 working-time budget, with lunch and breaks outside it. No pre-course work is required.

Use the daily sessions as flexible planning blocks. Most of the week is applied work, including setup and reporting; five video lessons form part of the required path. Resource durations help you judge the amount of material. Move between the steps of an exercise naturally, and put time saved by faster reading or an expert skip into practice.

Only skip when you can already demonstrate the resource's stated mastery criterion. Familiarity with a term or having used AI before is insufficient. Replace skipped instruction with a harder experiment and record one piece of evidence in your project notes.

Course policy: use one plan from each of two providers; use two plans from one provider only when it is strictly superior for your work. Reassess as the AI race changes. This dated example uses Claude Code for development and Codex for comparison/review: \$300 for one month, with a \$200 alternative. Cross-provider errors may overlap less, but agreement still needs evidence. Verify plans during Day 1.

Navigation: potential and progress pp 2–3; schedules and labs pp 7–17; reusable briefs and report pp 18–20; resources pp 21–31.

What you can build now

During Day 1's rediscovery session, read this primer, explore [R29](#), [R30](#) and [R31](#), then choose an idea to revisit.

Bring your experience and raise your ambition

Think of a useful tool, awkward integration or side project you have postponed because the implementation effort felt too large. Bring that idea into this week. Your knowledge of users, architecture and failure modes gives you a place to start; agent skills give you another way to turn that knowledge into working software.

In [R29](#), Simon Willison describes moving beyond feeling disheartened by AI coding. His point is encouraging for an experienced developer: knowing which problems matter, what is possible and how to judge a result remains valuable as implementation becomes easier to delegate.

See a wish become a working tool

Willison wanted a browser interface for a command-line GIF optimizer. In [R30](#), he shows the application, its animated demo and the prompts he sent from his phone. An agent handled the WebAssembly build and interface work; he supplied the intent, experience and follow-up decisions. Look for an equivalent opportunity in your own work: a small tool you would enjoy having, but have never made time to build.

Revisit yesterday's impossible task

[R31](#) gives a concrete reason to retest old assumptions. Steinberger reports that earlier models failed to rewrite a core part of VibeTunnel, while a later Codex run completed the conversion after more than five hours. His account spans changes within 2025; the following page adds measured capability progress into 2026. A previous failed attempt need not determine what you can do with today's setup.

These are firsthand examples of possibility, not promised completion times. Use their ambition to choose an experiment and use the course's checks to decide whether it worked.

Choose an ambitious experiment

List three ideas you postponed and what stopped you. Try a personal utility, a web app improvement or a fixture-based Web3 explorer. Pick the one you most want to exist.

Define the smallest useful demonstration and one acceptance check. Keep a stretch goal beside it: something you would attempt if implementation took much less of your time.

Record the old obstacle, your current hypothesis and the experiment you will run later today. Return to this note in the final report: what became possible, what still needed your judgment, and what will you try next?

What has changed and what has not

Agents can now complete substantially longer tasks in some measured settings. Read that progress alongside the developer stories: use it to reconsider what you attempt, then measure the benefit on your own project.

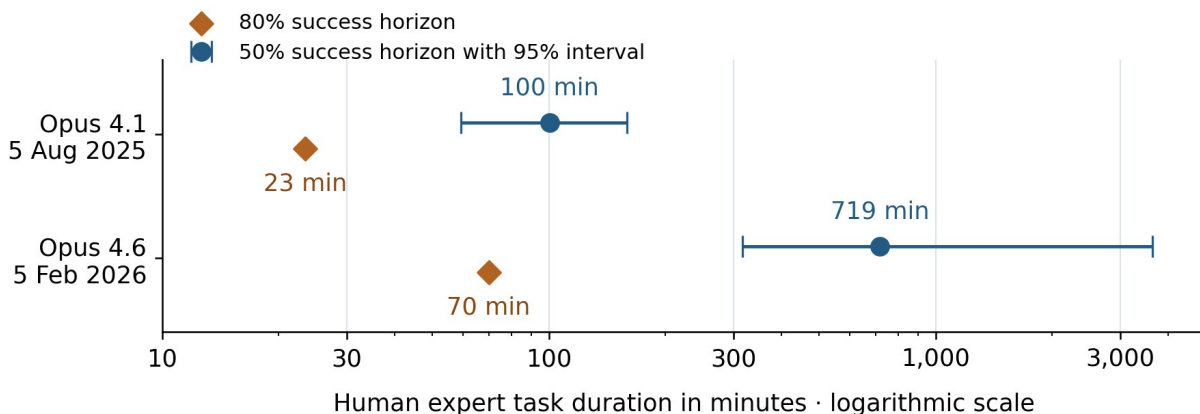


Figure 1. METR task horizons, same model family and listed react scaffold family in TH1.1. Human-expert task time, not agent runtime. Source: METR time-horizon dataset, last updated 8 May 2026; retrieved 16 September 2026. Shown intervals are reported 95% confidence intervals for the 50% horizon.

In this measured comparison, the 50%-success horizon rises from about 100 minutes for Opus 4.1 (August 2025) to 719 minutes for Opus 4.6 (February 2026): roughly 7.2×. The 80%-success horizon rises from about 23 to 70 minutes, roughly 3.0×. The wide intervals and difference between reliability levels matter. This is strong progress evidence, not a forecast of September performance or a promise of unattended production work. [METR](#)

Epoch's September analysis provides a second view of capability progress. Its index is not a developer-speed percentage. METR's February developer-productivity update shows why local measurement is still necessary: newer speedup estimates have uncertainty and selection problems. [Epoch trend](#) · [METR productivity update](#)

Habits to retest this week

- For a clear task, try an outcome, constraints and verifiable acceptance criteria. Add implementation steps when architecture, risk or observed failures justify them; concise prompts do not always win.
- Test whether mandatory planning, repeated self-check instructions, fixed context resets and artificial sprints improve the current model on your task. Retain them only when they help.
- Start with one capable agent. Add a worker when independence or isolated context has a specific benefit; more agents can also add cost, conflicting edits and integration work.

Anthropic's harness case study documents removing scaffolding as models improved. Its cost and timing observations are case-specific. The lesson is to simplify experimentally, not to abandon tests, context discipline or human judgment. [Harness case study](#)

Current model quality and value

Use this table to choose a starting point, then evaluate on your project. Artificial Analysis Coding Agent Index v1.5, retrieved 16 September 2026. These are model plus harness plus effort configurations; rounded scores do not establish a statistical tie.

Configuration	Index	API cost per task	Plan cost per task at capacity	Agent time
GPT 6 Astra · Codex · max	62	\$7.47	\$0.54 (\$0.22–1.45)	29.4m
Claude Fable 5.1 · Claude Code · max with fallback	62	\$12.39	\$2.48* (\$1.24–4.96)	34.8m
Claude Opus 5 · Claude Code · max	60	\$10.79	\$2.16 (\$1.08–4.32)	41.9m
GPT 5.6 Sol · Codex · max	55	\$6.58	\$0.48 (\$0.19–1.28)	20.6m
DeepSeek V4 Pro 0813 · Codex · max	43	\$0.24	Not included	40.3m
Gemini 3.8 Flash · Antigravity SDK · high	42	\$2.47	Not established	11.7m
DeepSeek V4 Flash 0731 · Codex · max	39	\$0.08	Not included	18.3m

Plan values are low-confidence estimates at full utilization; parentheses show sensitivity ranges. Pro 5x costs \$100/month; Max 20x, \$200. *Fable allocates half the fee to about 40 Fable-first attempts and uses the other half for about 46 Opus attempts. These are modeled capacities, not published quotas.

Source tables: [Codex comparison](#) and [Claude Code comparison](#). The index equally weights three evaluation families, with 303 tasks and three attempts per task. API token cost excludes subscription charges, supervision and production infrastructure; reported agent time excludes some overhead. [Methodology](#)

How to use the estimates

Astra and Fable are strong initial quality references. Astra has the lower measured API cost here. Lower-cost configurations merit testing on bounded work; lower API prices do not imply equivalent quality. Results depend on task mix and do not establish Solidity-audit competence.

Use a current flagship for the first rediscovery task, within included usage. Next compare a lower-effort or cheaper configuration on a repeated task. Route by observed acceptance and total work saved. Do not divide the composite index into price and call the result cost per success.

Keep a small evidence log

Log the starting commit, acceptance criteria, model/harness/effort, prompt/skills/workers, elapsed and intervention time, quota consumed, executed checks and rework. Compare spend and human review time per accepted task, alongside escaped defects. API benchmark cost per attempted task is a different measure.

A one-week comparison is a small convenience sample. Use it to choose your own setup provisionally, not to publish a universal model ranking. Recheck the live tables and repeat one representative task after major model or harness changes.

Plans and the working budget

Budget assumption: \$200–\$500 per developer per month, USD before tax and regional differences. If buying new access, budget a monthly charge per plan; existing subscriptions may already cover the course. The bundles below are current examples of the provider-allocation policy, not permanent provider endorsements.

Course operating policy: one plan from each of two providers. Different model families may catch different mistakes, although their errors can still overlap. Exception: two plans from one provider when it is strictly superior for the relevant work after considering accepted results, review/rework time, capacity and price. A small lead on one benchmark is insufficient. Reassess after major releases and at renewal; keep the error-diversity tradeoff explicit.

Route	Monthly purchases	List total
Start at \$200	Claude Max 5x \$100 + ChatGPT Pro 5x \$100	\$200
Recommended	Claude Max 20x \$200 + ChatGPT Pro 5x \$100	\$300
Authorized envelope	Recommended bundle plus up to \$200 reserved for measured capacity needs	Up to \$500

Prices: [OpenAI pricing](#), [Pro tiers](#) and [Claude plan guide](#). The reserve is not a purchase instruction. Request a larger budget before exceeding it. These subscriptions have usage limits; do not assume 36h30 of uninterrupted heavy parallel work.

As of 16 September: new ChatGPT Pro 20x purchases/upgrades are paused; existing 20x renewals and Pro 5x availability are unaffected. Check the linked Pro tier notice before changing a plan.

Check these during setup

- Sign in through the subscription route. Inspect the actual model and usage meters.
- Check both session and weekly limits. Claude use shares a pool across surfaces; current Max Fable inclusion is restricted rather than unlimited. Codex local and cloud work also shares allowance.

Operational details: [Claude subscription authentication](#) and [current Claude inclusion](#). Account-specific quotas and model availability take precedence over this snapshot.

What makes usage efficient

Start with clear inputs and a cheap, decisive verifier. Avoid irrelevant context, reuse stable procedures as skills, cap retries and lower effort only after measuring acceptance. A short expensive run can cost less overall than repeated cheap failures. Use subscriptions for ordinary frontier-model development.

Under normal circumstances, do not buy frontier-model API credits. If one plan runs out, switch to another; [cmux](#) helps manage the agent sessions (Day 1). Buy an additional plan when the value of the extra developer time it is expected to save exceeds its cost: $\text{hours saved} \times \text{value per hour} > \text{plan cost}$. Count review/rework time and use the same billing period. Follow the provider-allocation policy; request approval if total subscriptions would exceed \$500/month.

The concepts you will use

Concept	Working definition
Model	The system that reasons and generates actions. Quality and effort affect what it can do.
Harness	The surrounding runtime: tool access, instructions, state, permissions, feedback and execution loop. Claude Code and Codex are harnesses.
Tool or MCP integration	A callable capability or connection. A tool can provide data or cause side effects; its output may contain untrusted text.
Skill	Reusable procedural knowledge loaded when relevant, often a SKILL.md plus references or scripts. It does not itself guarantee correct execution.
Goal	A desired observable completion state, supported by executed evidence.
Loop	Repeated work or observation. It needs an exit condition and a bound, even when the goal sounds clear.
Workflow graph	Dependencies and control flow among steps or workers, including branches, joins and retries. This course does not mean a knowledge graph.
Ultra or ultracode	An effort and orchestration mode in a particular harness. It is not a universal model tier or a subscription plan.
Routine	A saved recurring or event-triggered workflow with an environment, inputs and an output contract.

Choose one small side project

Use something you want, but constrain it to 3–5 vertical slices. Good web options include a CSV import reviewer, a searchable personal reading queue, or a small issue triage dashboard. A read-only wallet activity viewer or event decoder makes a useful Web3 project. A contract exercise can use a toy escrow or vault on a local chain.

Your chosen project must run locally, use deterministic fixtures, and have an observable definition of done. Reserve one unseen bug or feature for Day 5. If setup or external integration is still blocked after its allotted time, use static fixtures and one local screen or CLI. The course does not require production deployment, real funds, a paid data API or an existing backlog.

Keep four lightweight artifacts

Maintain a one-page project brief, an acceptance/evidence list, a run-and-cost log, and a short next-step note. Plain Markdown files are enough. Commit project instructions and skills when a fresh checkout needs them. Keep secrets out of those files. These artifacts are produced inside the scheduled work, not as homework.

Day 1 Rediscover and establish a baseline

By the end: one ambitious idea made testable, one running slice, two harnesses that can inspect and test the repository, and a provisional primary setup.

Time	Study session
1h 30m	Orientation, potential, ambition and capability evidence · R29 , R30 , R31 , R01 , R02 , R03
45m	Quality, plans and field experience · R04 , R05 , R25
30m	Choose the project, scope and acceptance criteria
1h	Harness concepts, guides, installation and authentication · R06
1h	Rediscovery experiment on one bounded change
1h 45m	Implement the first vertical slice
1h	Compare the second harness, inspect results and update the evidence log
7h 30m	Planned working time

Open the resource cards for the assigned scope and expert-only skip criteria. Session budgets include reading, notes, transitions and practice; adjust the pace within each day.

Primer

Give the agent responsibility for a result it can verify. Your job is to supply missing context, meaningful constraints and an independent acceptance signal. Running more tools is useful only when it advances that result.

Practice for day 1 Rediscover and establish a baseline

Setup and rediscovery

- During setup, create a minimal runnable scaffold and fixture commit. Choose a task such as CSV validation, a pagination bug or deterministic event decoding. Define success before either run. Ask the agent to inspect the repository and available commands, then execute a harmless check. Verify subscription authentication and quota controls.
- Run two short attempts in separate repository copies and fresh sessions, from the same commit with the same model and effort: your genuine previous prompting approach, and an outcome brief with relevant context and acceptance checks. If you have no established approach, compare that brief with the same brief plus explicit implementation steps; either may win. Give the attempts comparable execution limits and leave room to inspect their artifacts. Do not ask either to reproduce a deliberately broken solution.
- Record interventions, accepted behavior, review time and usage. This experiment tests the whole prompt strategy on one task; it is not causal proof about every individual instruction.

Build and compare

Take one slice through implementation, execution, inspection and a small correction. A slice includes input, useful behavior and visible output: for example, import a fixture CSV, explain one invalid row and show valid rows. Compare the result with realistic human effort and defects; neither a bug nor a passing test settles whether AI helped.

For the second-provider comparison, use a separate repository copy and fresh session with the same starting commit and brief. Keep acceptance tests outside its write access and held-out answers outside its read access. Inspect shared agent memory; disable it for this comparison or record possible contamination. Include both the run and its review in the comparison session. Changing model and harness confounds their effects, so label this a setup comparison. Record unique mistakes, shared misses and reviewer findings; inspect test changes, leaked answers or other benchmark contamination.

Evidence to keep

A reproducible startup command, a passing behavior check, a screenshot or sample output, the relevant diff and two logged comparisons. If a task fails, a narrow reproducible failure and diagnosis is a valid training result.

Move between subscription sessions

During harness setup, practise a handoff: stop the first writer, save the relevant commit or diff and a short status note, then open the other subscribed agent with the brief and acceptance evidence. Keep one active owner for each editable worktree. Each agent retains its own subscription login and quota.

Recommended for multitasking on macOS: [cmux](#). Keep Codex CLI and Claude Code in labeled workspaces or split panes; configure notifications to know which agent needs attention. In the existing setup block, run two independent tasks in separate Git worktrees, then inspect both results. Each agent keeps its own login and quota; switching providers uses the handoff above. Reading: [cmux Getting Started](#), about 5 minutes (Q5/U5/S5.00), as part of harness setup. Skip the reading only if deeply familiar with configuring cmux and managing isolated agent sessions, notifications and handoffs. The native app requires macOS 14+; use separate terminal windows if unavailable.

Day 2 Prompt and grill for correctness

By the end: a brief with fewer hidden assumptions, a feature checked independently of the agent’s claims, and a justified choice of development mode.

Time	Study session
1h 30m	Primer, grilling, prompting and visual constraints · R07 , R08 , R09
45m	Verification, tool access and development mode · R10 , R11 , R32
45m	Practice all three directions of grilling
1h 30m	Implement a feature from the clarified brief
1h	Inject and diagnose a failure
1h	Compare visual or interface alternatives
30m	Choose the development mode, complete checks and prepare the handoff · R32
7h	Planned working time

Open the resource cards for the assigned scope and expert-only skip criteria. Session budgets include reading, notes, transitions and practice; adjust the pace within each day.

Primer

A useful prompt names the outcome, relevant context, constraints, available actions and acceptance evidence. Grilling turns uncertainty into explicit decisions. It should stop once the remaining choices are safe to make locally.

Practice for day 2 Prompt and grill for correctness

Three directions of grilling

- Agent interviews you: give a deliberately ambiguous feature request. Ask for up to five high-impact questions, one at a time, with proposed defaults and the consequence of each choice. Answer before implementation; if material uncertainty remains after five questions, resolve it or defer that part of the feature. State explicitly when backward compatibility is unnecessary, and name abstractions or extra features you do not need.
- Agent challenges the design: ask it to identify fragile assumptions, a simpler alternative, failure modes and the cheapest experiment that would change the decision.
- You interrogate the agent: ask which command actually ran, which artifact supports its claim, what remains untested and what evidence would falsify its conclusion. Do not request hidden chain-of-thought; request inspectable reasoning and evidence.

Make verification capable of failing

For the web track, create a user-behavior check, then deliberately change product behavior—for example, accept an invalid CSV row or display a wrong balance. Confirm the unchanged check detects that defect, then restore correct behavior. For a contract track, explicitly set `fail_on_revert=false`, remove required funding or approval from the fixture, and observe whether an invariant campaign passes without meaningful state changes. Use a transition counter to expose this false assurance, then restore the fixture. Count successful transitions, not attempts.

For Web3, add one adversarial economic scenario: who can profit, avoid a cost, delay settlement or impose costs on others? State assumptions about fees, timing, permissions and coordination; make a local reproducer or invariant where possible. Also check relevant chain, decimal and revert behavior. Inspect whether generated tests use an oracle independent of the implementation.

Iterate with references

Ask for two or three interface alternatives in isolation, using realistic empty, loading and error states. You own taste: require that a new user can identify the next action, recover from an error and distinguish real from placeholder data. Choose against references, hierarchy and copy, then integrate one option. Ask the agent to remove unnecessary abstractions, duplicate logic and speculative compatibility. For this exercise, use a fresh reviewer session; the implementing agent should still run its own checks, and you own acceptance. Budget hands-on beta testing even when code arrives quickly.

Choose a development mode

During the handoff session, use the [R32](#) exercise to classify the change by criticality and defect detectability. Record who writes the code, how you will check it and when review is needed. Apply that decision and prepare the handoff. Record unfinished checks or required review; elapsed time does not make a change ready to accept.

Day 3 Build skills and bounded goal loops

By the end: one tested skill and a goal run that can recover without losing its acceptance criteria.

Time	Study session
1h	Primer, skills demonstration, format and evaluation · R12 , R13 , R14
1h	Long-running harnesses, goals and loops · R15 , R16 , R17
1h 15m	Create and test one project skill
45m	Clean up context and project instructions
1h 45m	Complete a longer feature with a goal
1h 15m	Interrupt and recover, inspect evidence and cost, then stop recurring work
7h	Planned working time

Open the resource cards for the assigned scope and expert-only skip criteria. Session budgets include reading, notes, transitions and practice; adjust the pace within each day.

Primer

Use skills for repeatable decisions and procedures, not as a dumping ground for everything the agent might need. Keep project-wide rules short. Put large examples and reference material behind explicit links so they are loaded when useful.

Practice for day 3 Build skills and bounded goal loops

Create a skill from yesterday's actual work

Choose a procedure you repeated: reproducing a UI bug, validating an import, checking event decoding, or verifying a contract invariant. Include when to use it, required inputs, ordered actions, the expected evidence and a clear stopping or escalation condition. Store it under the project location described by the runtime documentation.

As a small smoke test, try three prompts: one where it should activate, a similar one where it should not, and a boundary case with a missing input. Check the loaded context and resulting artifact. Fix an overbroad trigger or missing instruction. Three cases do not establish reliability; add representative cases as the skill becomes important. Do not merely ask the agent whether its own skill is good.

Run a longer goal

Choose a feature that needs several edit-execute-inspect cycles. Express done as observable behavior with concrete checks. Set a practical deadline and a finite retry budget; use actual runtime/tool controls when available. A natural-language limit is not an enforced runtime limit. Save a brief progress note before interrupting the session, then resume or start fresh from that note. Within this block, start one read-only local /loop to inspect a changing log or task status. Observe two useful iterations, then stop it; do not let the loop edit the same files as the goal worker.

Current Claude goals use transcript evidence for evaluation. Require the harness to run checks and show outputs; a confident summary is insufficient. Force one failed check, repair the cause, and distinguish actual success from an empty or irrelevant test run.

Retest one old workaround

Remove or simplify one inherited instruction—such as a compulsory lengthy plan for a trivial edit, redundant self-checking or arbitrary reset threshold. Compare the result on a bounded task. Keep any constraint that still prevents a demonstrated failure. Stop local recurring checks before leaving the exercise. Start a fresh session for a distinct task when prior context adds noise, carrying forward only the relevant brief and evidence. Test this on your work rather than impose a universal reset size.

Day 4 Coordinate agents and parallel work

By the end: a workflow graph, a bounded recovery, and a measured decision about how many agent workstreams you can supervise effectively.

Time	Study session
1h 45m	Primer, graphs, recovery, Ultra, isolation and effort · R18 , R19 , R20 , R21 , R22
30m	Draw a graph and define its contracts
2h	Run a small parallel workflow
1h	Force an integration or branch failure
1h	Compare and simplify effort or orchestration
1h	Multitasking primer, readings and two-workstream practice · R26 , R27 , R28
45m	Save the workflow and review provenance
8h	Planned working time

Open the resource cards for the assigned scope and expert-only skip criteria. Session budgets include reading, notes, transitions and practice; adjust the pace within each day.

Primer

A workflow graph expresses dependencies: what can proceed independently, what must wait, and what happens when evidence fails. A fixed graph is useful when the process is known; an agent-generated workflow can adapt its branches. Both need a clear owner for integration.

Practice for day 4 Use graphs and Ultra selectively

Use a small graph

For this introductory lab, use at most two concurrent workers and one integrator. Commit the shared data contract, then verify that both worker worktrees start from that exact revision. Implement the UI and adapter separately. Require changed files, executed checks and open assumptions at the join. Isolate concurrent writers; even review commands may create caches or test artifacts. During a long run, try one other independent task and leave a return note. The two-context limit is a training choice, not a demonstrated optimum. Use cmux on macOS to monitor the separate agent sessions and notifications; each writer still needs its own worktree. Switch plans when included usage runs out.

Force one branch failure and record which workers actually rerun. Claude workflow relaunch replays agent start order: a failed agent and all agents started after it run again, including completed siblings. Selective reuse requires explicit artifact tracking or separately managed stages. Inspect worker and retry bounds; size guidance and prompt requests are not hard caps. See [R19](#).

Operate the current feature

In current Claude Code, `/effort ultracode` combines extra-high reasoning with automatic workflow orchestration for substantive tasks; `/effort high` exits that mode. Inspect runs with `/workflows` and practice pause, stop and recovery. A keyword invocation in an interactive prompt is not equivalent to persistent effort configuration, and the keyword does not trigger the same behavior in noninteractive routines or webhooks. Follow [R20](#) for your installed version.

If the feature is unavailable for your account/version, perform the same dependency graph with ordinary subagents and explicit joins. The learning objective is control and evidence, not a particular switch. Do not spend extra simply to reproduce the branding.

Measure whether it helped

Within the 60-minute comparison, first compare the same capable model at two effort levels on a bounded task. If time permits, try a strong model for specification and a cheaper capable model for implementation. Treat routing and overengineering concerns as hypotheses: compare accepted quality, complexity, review time and usage. Do not assume maximum effort wins, or that lower effort harms audits. Record what replay did after the failed branch.

Optional substitute in that same 60-minute block: have an isolated agent reimplement one small pure function from the behavior specification, without the original code. Compare both against held-out cases. Disagreement reveals a question to investigate; agreement does not prove correctness. This explores work that may be affordable with agents but too expensive to commission manually.

Manage several agent workstreams

During Day 4’s multitasking session, read this primer and [R26](#), [R27](#) and [R28](#), then practise managing two independent workstreams.

From supervising an intern to managing a department

For your first training tasks, treat one agent like an intern: give very explicit instructions, explain context and constraints, and check its changes, commands, tests and results. Learn where its work is reliable and where your instructions leave gaps.

As you become proficient, act like a department manager with multiple AI workers. Set priorities, delegate outcomes, assign owners, coordinate dependencies and review acceptance evidence. Give workers room to choose implementation details. Use [R32](#) to choose code authorship and review depth.

Stage	Your main responsibility	Agent responsibility
Starting	Give explicit instructions; check changes, tests and results.	Complete one bounded change and its checks.
Developing	Delegate two independent tasks; manage ownership, handoffs and review.	Work in isolated sessions and report blockers.
Advanced	Prioritize multiple workers; delegate outcomes; coordinate dependencies and acceptance.	Execute and coordinate delegated work within those boundaries.

Use the waiting time

Longer autonomous tasks leave more time to advance other work while an agent runs. Steinberger describes roughly 30-minute runs and refactors scheduled around other activities ([R26](#)); Osmani combines close supervision with asynchronous delegation ([R27](#)). Choose concurrency based on task independence, review capacity and switching overhead ([R28](#)).

Keep your own attention on one decision at a time. While one agent runs, prepare the next brief, inspect a completed result or work on an independent task. In cmux, label sessions by outcome and respond to meaningful notifications. Parallel sessions still share each provider’s plan limits.

Manage two independent workstreams

Choose two small independent tasks, such as a CSV validation fix and a fixture-based event-decoding improvement. Give each a brief, acceptance check and separate worktree. Record owner, state, next action and evidence in a small task list.

Launch the first agent, then use its execution time to start and supervise the second. Mark each task running, blocked or ready for review. Resolve consequential questions promptly; avoid repeatedly checking a session that has nothing new to report.

Inspect both outputs and integrate only accepted changes. Log overlapping execution, active human time, review/rework and accepted results. Record whether parallel work helped; unfinished work remains unfinished. Stop or hand off active runs explicitly.

Increase concurrency gradually after you can preserve review quality and ownership. If completed work queues faster than you can evaluate it, pause new starts. Use closer supervision again for unfamiliar or consequential decisions; the department-manager perspective includes knowing when to intervene.

Day 5 Turn a proven process into a routine

By the end: a repeatable routine, a small automation boundary and a report that shows what you learned.

Time	Study session
1h	Primer, durable routines and programmatic use · R23 , R24
1h 15m	Test a routine on repeated and changed inputs
45m	Build one small CLI wrapper
2h 30m	Capstone task including independent review
1h	Write your personal operating plan and short training report
30m	Clean up schedules, permissions and spending
7h	Planned working time

Open the resource cards for the assigned scope and expert-only skip criteria. Session budgets include reading, notes, transitions and practice; adjust the pace within each day.

Primer

Automate a procedure after you can recognize a useful result. A successful run status means the infrastructure completed; it does not prove that the requested development outcome is correct. Give every routine an artifact contract and an independent check.

Practice for day 5 Turn a proven process into a routine

Routine exercise

Choose a low-impact job: triage fixture issues, inspect a test-failure log, or summarize changed dependencies into a local report. Define trigger, input version, permissions, output path, validation, owner and failure behavior. Run twice on identical input, then once with a changed fixture inside the block. The second identical run must not duplicate effects. Missing or invalid output must be visible.

Claude Code cloud routines can run without your laptop and without per-run approval. Use a training repository, minimal connector access and committed skills. If durable scheduling is unavailable, produce and manually execute the same routine contract through the CLI, then document the missing feature. Disable any live schedule at the end unless you deliberately choose to retain it.

Small building section

In the inspected training repository, wrap the supported CLI in a short script: accept a task input, invoke the agent with structured output, enforce an outer timeout, parse the result and independently verify a required artifact. Demonstrate a success and a malformed or failed result. Use ordinary subscription authentication and the existing CLI for this exercise.

Capstone and evidence

Take the held-back bug or feature from a fresh brief to an accepted change. Include clarification, implementation, executed checks, a fresh evaluator session and human inspection in the capstone. Under the default policy, use the other provider; under the justified same-provider exception, use a separate session. Give the fresh reviewer the task, diff and acceptance criteria, and require its own checks. As an optional experiment, withhold the implementer's rationale until its first pass, then share necessary design context. This explores possible anchoring; it is not a proven universal review order. Inspect evaluation contamination. Narrow any unfinished slice and report it accurately.

Finish the report and operating plan below. Inspect active routines, temporary credentials, worktrees and actual spending during the final cleanup. The course ends with a maintainable project and evidence of judgment, not a requirement to publish or deploy.

A graph with evidence and recovery

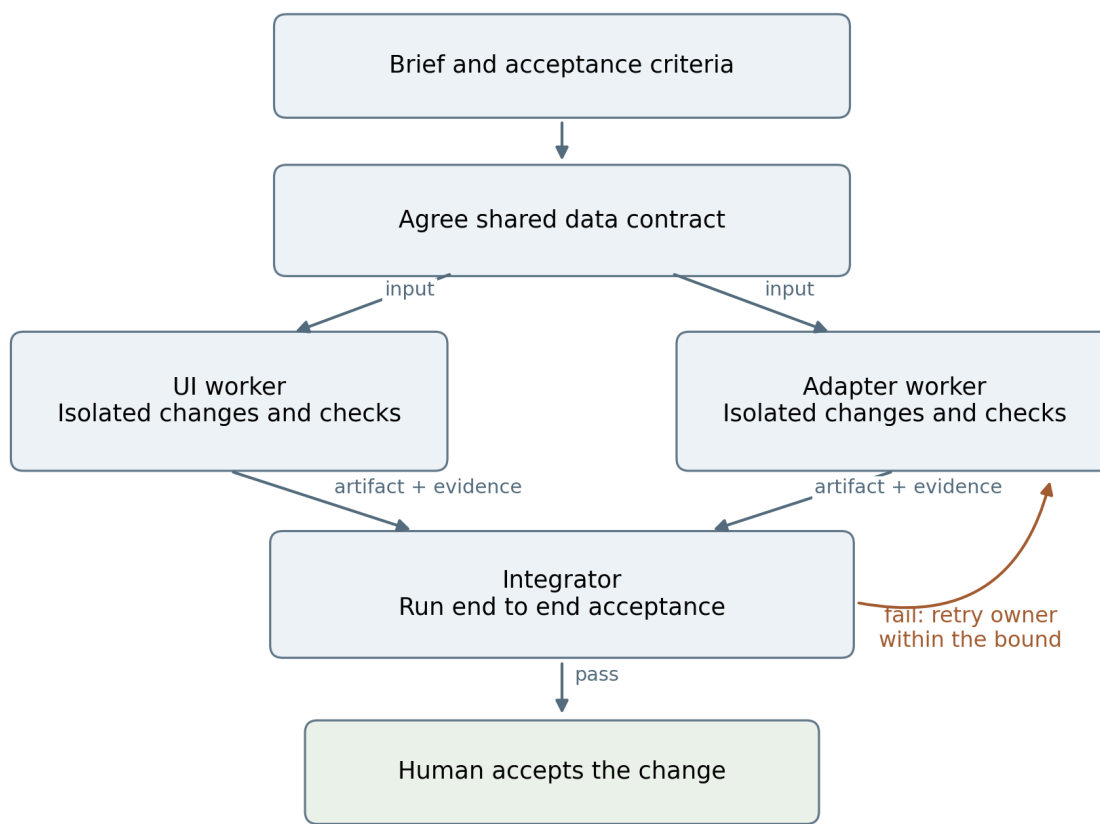


Figure 2. A small development workflow. A worker returns artifacts and evidence; the integrator owns acceptance. Reuse requires unchanged inputs and complete dependencies; native runtime replay may rerun additional workers.

Specify data, ordering and shared-resource dependencies before running the graph. “Adapter ready” should mean a file or commit plus a contract check, not a worker saying it is done. At the join, inspect assumptions and run the integrated behavior. Branch completion and system completion are separate claims.

These are introductory lab limits, not measured optima: use at most two concurrent workers, one retry per failed branch and a 90-minute execution deadline inside the 120-minute block. Keep the remaining time for inspection. Enforce the limits in the workflow/runtime or by active supervision; do not mistake a size suggestion for a hard cap.

Stop when there is no new evidence after repeated attempts, the scope changes materially, required access is missing or the spending boundary is reached. Preserve the current artifacts and a minimal reproducer. Have the agent ask a focused question or propose a smaller accepted slice.

Knowledge graphs describe entities and relationships in data. This graph describes control flow and artifact dependencies.

Briefs and questions to reuse

A coding outcome brief

Goal: [observable user or system result].

Context: [repository, relevant paths, existing behavior, reproducer or reference].

Constraints: [compatibility, architecture, scope and actions requiring my decision].

Acceptance: [behavior checks, edge cases, expected artifact and evidence].

Process: inspect what matters; ask about high-impact ambiguity; choose a proportionate plan; implement and run the checks.

Stop: [deadline, bounded retries or blocked prerequisite]. Return changed files, executed checks and unresolved risks.

Use only fields that add information. For a tiny, clear change, a short request plus an exact file and verification step may be enough. Do not force planning or lengthy explanations on every edit.

Agent interviews the developer

Before implementing, interview me about the uncertainties that could change the design or acceptance criteria. Ask one high-impact question at a time, at most five. Offer a recommended default and explain its consequence. Then summarize the decisions and remaining assumptions. Resolve or defer any consequential uncertainty still open after the five-question training limit.

Agent challenges the design

Review this proposal for fragile assumptions and avoidable complexity. Identify the strongest simpler alternative, likely failure modes and one cheap experiment that could change our choice. Tie each concern to a concrete consequence; stop after the material issues.

Developer checks the agent

What evidence supports completion? List the commands actually executed, the relevant outputs and any untested behavior. Which check would fail if your implementation were wrong? Identify one assumption I should independently verify.

A worker contract

Task and input: [one bounded deliverable and source revision].

Ownership: [allowed files or read-only scope]. Dependencies: [required artifact].

Return: changed files or commit, checks executed, output artifact and open assumptions.

Do not claim completion without the required evidence. Stop after [bound] and return a reproducible blocker if needed.

The report and the operating plan

Short training report

Write 400–700 words during the reporting session. Include links to artifacts or commits; a manager should be able to see that you used and evaluated the process. There is no exam and no requirement that every experiment succeed.

- Project and accepted result: what changed, how it was checked and what remains unfinished.
- One rediscovery: an old belief or workflow you tested, the comparison and whether you changed your mind.
- Your current setup: providers, models, harnesses, effort and plans; actual spend/quota observations and why this choice fits your work.
- Process evidence: one grilling decision, one tested skill, one goal or recovery, one graph/Ultra decision, one routine result and one multitasking decision. Link to small artifacts rather than paste transcripts.
- Failure and next step: a wrong claim or failed check you caught, how you caught it and one adjustment for the next month.

Personal operating plan

Write a half-page default process alongside your training report. State your two-plan allocation, primary model and reviewer, and evidence supporting any same-provider exception. Describe task-based model/effort routing, project rules versus skills, when graphs help, required acceptance evidence and budget escalation. Set a concurrency limit and explain how you will clear pending reviews before starting more work.

Choose a brief weekly maintenance habit: inspect failed runs, unique reviewer findings, shared misses and actual usage; remove stale instructions and improve a repeated procedure. After major releases and at each plan renewal, reassess both provider allocation and effort. Retain two providers unless one is strictly superior for your work. Record the next review date.

Evidence log template

Field	Record
Task and starting state	Brief, fixture or issue; commit; predetermined acceptance criteria
Configuration	Model; harness/version; effort; skills; workers
Cost and time	Elapsed minutes; intervention/review minutes; quota consumed and allocated subscription cost
Evidence	Commands and outputs; screenshot or artifact; accepted/rejected
Learning	Failure caught; rework; next adjustment

For Web3 work, record the chain/environment and fixture block. Apply the [R32](#) development policy to each component. For critical logic with hard-to-detect defects, write the implementation by hand and use AI to hunt for bugs and exploits, with tests and security review proportionate to impact. [R25](#) explains a practitioner's preference for human authorship of core code. General coding scores do not establish economic-reasoning or vulnerability-search performance.

Resources R01, R02, R03

Each resource card gives an approximate study duration and an expert-only skip condition. Read or watch only the named scope. These estimates describe the amount of material, not a countdown for each action. Q is instructional quality; U is current applicability; both are out of 5. $S = 0.65Q + 0.35U$. These are editorial resource ratings, not model scores. Inline citations elsewhere are evidence links, not additional assignments.

R01 What longer task horizons mean

Estimated study time: about 10 minutes • Q 5 / U 4 / S 4.65

[METR time horizons](#) (about 10m)

Read the definition, reliability levels, chart and uncertainty explanation. Use the dated comparison in this guide; do not infer September results from the May dataset.

Skip only if you can distinguish human task duration from agent runtime, explain 50% versus 80% reliability, confidence intervals and scaffold dependence.

R02 Rediscover what the current agent can do

Estimated study time: about 20 minutes • Q 4.5 / U 4.5 / S 4.50

[Theo Chu: The capability curve](#) (19m22s)

Watch the full talk, then name one inherited instruction to retest. Focus on fresh task evaluations, useful tool access and obsolete scaffolding. The benchmark evidence is in [R01](#) and the guide's progress figure.

Skip only if you have recently compared old and current agent setups on representative tasks and can show which inherited instructions you removed, retained or changed based on results.

Session: [The capability curve, Code with Claude Tokyo](#). One live demo encounters a connectivity failure; its intended result is described rather than demonstrated.

R03 Measure real developer productivity

Estimated study time: about 15 minutes • Q 5 / U 4 / S 4.65

[METR developer uplift update](#) (about 15m)

Read the results and selection-bias discussion. Contrast task capability with experienced developers working in their own repositories.

Skip only if you can explain the early slowdown finding, why later estimates are inconclusive, and design a comparison that includes review and rework.

Resources R04, R05, R06

R04 Current coding quality and API value

Estimated study time: about 15 minutes • Q 4.5 / U 5 / S 4.68

[AA Codex comparison](#) (about 5m) • [AA Claude Code comparison](#) (about 5m) • [AA methodology](#) (about 5m)

Inspect only the coding-agent tables and methodology. Compare score, cost and time together; record the version and retrieval date.

Skip only if you can reproduce a current model-harness-effort comparison and explain its task mix, cost exclusions and limits for your project.

R05 Subscriptions and usage limits

Estimated study time: about 20 minutes • Q 5 / U 5 / S 5.00

[OpenAI pricing and plan limits](#) (under 5m) • [ChatGPT Pro tier notice](#) (under 5m) • [Claude plan guide](#) (under 5m) • [Claude current pricing and inclusion](#) (under 5m) • [Claude Code subscription authentication](#) (under 5m)

Read the subscription sections on included usage, shared limits and signing in. Apply the two-provider default and same-provider exception on page 5. Use another subscription when quota runs out. Check the linked official Pro-tier notice: new Pro 20x purchases and upgrades are currently paused; Pro 5x is available. Review your selected plans' limits and calculate how much time an additional plan must save to justify its cost.

Skip only if deeply familiar with signing into both subscribed agents, inspecting all relevant usage limits, justifying the provider allocation and handing work to another agent when a plan runs out.

R06 Understand and install the harness

Estimated study time: about 20 minutes • Q 5 / U 5 / S 5.00

[How Claude Code works](#) (about 10m) • [Claude Code quickstart](#) (under 5m) • [Codex quickstart](#) (under 5m)

Read the agent loop, tools, context and session behavior; skim both installation guides. Then apply the guides during the harness setup session.

Skip only if you can explain the model/tool/loop boundary, inspect active context and permissions, and install and authenticate both current harnesses without a tutorial.

Resources R07, R08, R09

R07 A worked example of grilling

Estimated study time: about 30 minutes • Q 4.8 / U 4.3 / S 4.63

[Matt Pocock: grilling demonstration, 12:16–21:59](#) (9m43s) • [Specification handoff, 30:36–36:32](#) (5m56s)

Watch these two excerpts (about 16 minutes total). Then draft and answer the highest-impact questions for your project. The full 96-minute workshop is not assigned.

Skip only if you routinely elicit hidden requirements, challenge alternatives and convert answers into falsifiable acceptance criteria without endless interviewing.

If video playback is unavailable, read the [timestamped transcript](#) for the same passages and use the saved time for practice. Test context-reset and testing choices on your own project.

R08 Prompt for coding outcomes

Estimated study time: about 25 minutes • Q 5 / U 5 / S 5.00

[Claude Code best practices](#) (about 25m)

Read verification, concrete context, exploration versus planning, course correction and context management. Avoid rereading the interview example already covered in [R07](#). Adapt the brief in this guide.

Skip only if you can give a minimal sufficient brief, choose when planning pays off, provide a reproducer or visual reference, and correct an agent using evidence rather than repeated vague requests.

R09 Give the agent visual constraints

Estimated study time: about 20 minutes • Q 4.5 / U 5 / S 4.68

[Matthew Dailey on designing with AI](#) (about 10m) • [Refero styles reference](#) (about 10m)

Read Dailey's design process, then browse only enough references to choose one visual direction. Write constraints, use existing components and compare 2–3 isolated alternatives.

Skip only if you already supply realistic preview states, design constraints and references, then compare screenshots against concrete criteria while keeping experimental variants out of production code.

If X is unavailable, read the [author's mirror](#) instead.

Resources R10, R11, R12

R10 Build an independent verification signal

Estimated study time: about 20 minutes • Q 5 / U 5 / S 5.00

[Web track: Playwright best practices](#) (about 20m) · [Contract track: Foundry invariant testing](#) (about 20m)

Choose ONE track. Web: behavior, isolation, locators, assertions and traces. Contracts: handlers, ghost state, revert policy, targeting and call summaries. Read-only Web3 apps can use the web track.

Skip only if web: you can write reliable behavior checks and use traces to diagnose failure. Contracts: you can build a stateful campaign, verify successful transitions and reproduce a failing sequence. You must meet the criterion for your chosen track.

R11 Keep tool access deliberate

Estimated study time: about 15 minutes • Q 5 / U 5 / S 5.00

[Claude Code security](#) (about 15m)

Read permissions, sandboxing and prompt-injection defenses. Apply the concepts to repository text, tool output, secrets and external actions.

Skip only if you can distinguish untrusted data from instructions, demonstrate least-privilege tool access, and identify which actions need a human decision in your workflow.

R12 See a repeated workflow become a skill

Estimated study time: about 20 minutes • Q 4.8 / U 4.5 / S 4.70

[Elie Schoppik: Why Use Skills I](#) (11m16s)

Watch the complete original lesson. Then identify a repeated project workflow, its trigger, inputs and checkable output. This introduces the process; [R13](#) supplies the format and [R14](#) supplies evaluation.

Skip only if you can already turn a repeated workflow into a skill, explain selective loading, invoke it reliably and verify its output in a real project.

Follow current runtime documentation for UI and tool behavior.

Resources R13, R14, R15

R13 The portable skill format

Estimated study time: about 15 minutes • Q 5 / U 5 / S 5.00

[Agent Skills specification](#) (about 15m)

Read directory structure, required frontmatter, name/description constraints and progressive disclosure. Treat optional tool declarations as runtime-dependent.

Skip only if you can author a valid SKILL.md with precise triggers and supporting files, and explain which fields are portable versus runtime-specific.

R14 Evaluate a custom skill in its runtime

Estimated study time: about 15 minutes • Q 4.8 / U 4.5 / S 4.70

[Creating Custom Skills, 05:10–16:15](#) (11m05s) • [Current Claude Code skill controls](#) (about 5m reading)

Watch the examples and evaluation portion only; [R13](#) already covers the repeated structure introduction. Then look up project discovery, invocation and permission controls in current docs. Distinguish valid structure from a skill that actually works.

Skip only if you can test intended and unintended activation, verify generated files and script behavior, inspect loaded context and configure explicit invocation and permissions for consequential actions.

R15 Understand the harness around a longer-running agent

Estimated study time: about 25 minutes • Q 4.7 / U 4.5 / S 4.63

[Lance Martin: Claude for long-horizon tasks, 00:00–19:31](#) (19m31s)

Watch the main talk; stop before Q&A at 19:31. Then identify the evaluator, durable state, sandbox and feedback in your own harness. Choose one inherited workaround to test, rather than copy the demonstrated architecture wholesale.

Skip only if you can distinguish model capability from harness effects, design a useful completion signal and explain when persistent state, isolated judging or extra scaffolding helps on your tasks.

Reported demonstrations and outcomes are case studies, not general productivity estimates.

Resources R16, R17, R18

R16 Use goals with real evidence

Estimated study time: about 15 minutes • Q 5 / U 5 / S 5.00

[Claude Code goals](#) (about 15m)

Read setting, completion, clearing and failure behavior. The goal evaluator relies on transcript evidence; it does not independently execute your test suite.

Skip only if you can define an observable completion condition, require executed evidence, identify false completion and stop or clear an unproductive goal.

R17 Separate repetition from completion

Estimated study time: about 15 minutes • Q 5 / U 5 / S 5.00

[Claude Code scheduled tasks and loops](#) (about 15m)

Read /loop scheduling, session lifetime and self-paced behavior. Contrast this local mechanism with the durable routines introduced on Day 5.

Skip only if you can choose a loop interval, explain session dependence, avoid overlapping work, and stop a recurring check once its purpose is met.

R18 Understand workflow graphs

Estimated study time: about 15 minutes • Q 4.5 / U 5 / S 4.68

[LangGraph workflows and agents](#) (about 15m)

Read prose and diagrams for parallelization, routing, orchestrator-workers and evaluator-optimizer. Skip framework installation and code blocks for this using-first course.

Skip only if you can draw dependencies, fan-out, join, routing and bounded retry, and explain when a single agent is simpler. You distinguish workflow graphs from knowledge graphs.

Resources R19, R20, R21

R19 Understand what recovery actually replays

Estimated study time: about 15 minutes • Q 4.8 / U 5 / S 4.87

[LangGraph checkpoints](#) (about 5m) · [Claude workflow recovery](#) (about 5m)

Read LangGraph's opening and fault-tolerance/pending-write bullets, then Claude's resume section. Predict replay after one failed parallel worker. No framework installation is required.

Skip only if you can distinguish checkpoint recovery, start-order replay and invalidation after changed inputs, and predict which completed workers each runtime reuses.

LangGraph can preserve successful nodes in a failed superstep with checkpointing. Claude may rerun later completed workers. Neither statement guarantees safe reuse after arbitrary input changes.

R20 Run and inspect Ultra workflows

Estimated study time: about 30 minutes • Q 4.8 / U 5 / S 4.87

[Claude Code workflows](#) (about 30m)

Read ultracode entry/exit, workflow monitoring, size guidance, pause/stop controls and saving scripts. [R19](#) already covers relaunch semantics. Check current availability/version in your installation. Skip the recovery section already read in [R19](#).

Skip only if you can distinguish model, reasoning effort and orchestration; run a small workflow; inspect workers and token use; stop it; and recover or save it without losing provenance.

R21 Give subagents bounded jobs

Estimated study time: about 20 minutes • Q 5 / U 5 / S 5.00

[Claude Code subagents](#) (about 20m)

Read delegation, tool access, fresh versus forked context and worktree isolation. Verify the worker's base revision. Forks can inherit the full conversation; a separate subagent is not automatically an independent reviewer.

Skip only if you can assign nonoverlapping work, isolate concurrent writes, specify an output schema, and integrate and test results without treating worker agreement as proof.

Resources R22, R23, R24, R25

R22 Tune effort and cost experimentally

Estimated study time: about 15 minutes • Q 4.8 / U 5 / S 4.87

[Anthropic cost and intelligence guidance](#) (about 15m)

Read prompt simplification, effort tuning and cost against outcomes. Include the comparison of a frontier model at lower effort with cheaper-model routing. Treat the provider's findings as a starting hypothesis for your tasks.

Skip only if you can compare accepted tasks across effort settings, include review/rework cost and recognize when a cheaper token rate increases total task cost.

R23 Routines that produce useful artifacts

Estimated study time: about 25 minutes • Q 5 / U 5 / S 5.00

[Claude Code routines](#) (about 25m)

Read triggers, repositories, connectors, environment, limits and troubleshooting. Notice fresh checkouts, committed skills and the distinction between run completion and task success.

Skip only if you can build an idempotent routine with minimal credentials, validate its output independently, prevent duplicate effects and handle failures and daily limits.

R24 A small amount of agent building

Estimated study time: about 20 minutes • Q 5 / U 5 / S 5.00

[Run Claude Code programmatically](#) (about 20m)

Read noninteractive invocation, structured output, exit status and session continuation. Ordinary CLI print mode can use subscription login; --bare does not use that login. Inspect project hooks and MCP configuration before running noninteractive tasks.

Skip only if you can wrap a supported CLI safely, parse a structured result, enforce a runtime limit and validate an artifact independently of the agent's success claim.

R25 Field experience from development and security work

Estimated study time: about 10 minutes • Q 4 / U 5 / S 4.35

[Additional development field notes](#) (about 5m)

Read the full notes, then choose one observation to test this week. Focus on design/implementation model routing, scarce premium usage, taste, code complexity, independent evaluation and useful work during agent latency.

Skip only if you already understand these field observations and can demonstrate a task-specific routing policy, an isolated evaluator, a complexity check and a comparison that records shared versus unique model errors.

These notes offer one practitioner's hypotheses. Their model comparisons are anecdotal; test claims about routing, effort, complexity and error diversity on your own tasks. Noncoding experience does not establish coding quality.

Resources R26, R27, R28

R26 Use agent wait time productively

Estimated study time: about 5 minutes • Q 4.5 / U 3.5 / S 4.15

[Peter Steinberger — Shipping at Inference-Speed \(28 Dec 2025\)](#)

Read “My Workflow” only. Focus on concurrent projects, roughly 30-minute autonomous runs, queued work and refactors scheduled during other activities. Treat the model preferences as historical; apply the current comparison in this guide. Keep separate worktrees for concurrent writers in this course.

Skip only if deeply familiar with shipping parallel-agent work and able to explain, from your own runs, how waiting time, task independence and review capacity affect throughput.

R27 Manage outcomes across agents

Estimated study time: about 10 minutes • Q 4.8 / U 4.5 / S 4.70

[Addy Osmani — Your AI coding agents need a manager \(8 Jan 2026\)](#)

Read “A mental model for the new workflow”, “Delegation”, “Async check-ins”, “The hard parts are real” and “A simple operating system for orchestration”. Apply the management practices: assign outcomes and ownership, identify dependencies, unblock work and control integration.

Skip only if deeply familiar with managing several agent workstreams and able to show explicit ownership, escalation rules, acceptance evidence and an integration order.

R28 Find a sustainable concurrency level

Estimated study time: about 5 minutes • Q 4.6 / U 4.8 / S 4.67

[Addy Osmani — Your parallel Agent limit \(7 Apr 2026\)](#)

Read the full article. Focus on scope per task, switching costs, review capacity and the author’s changes to his workflow. Choose your own limit from observed results; his preferred agent count is a personal example.

Skip only if deeply familiar with parallel-agent supervision and able to detect a growing review queue, reduce active work and measure accepted outcomes after rework.

Resources R29, R30, R31

R29 Your development experience still matters

Estimated study time: under 5 minutes • Q 4.3 / U 5 / S 4.55

[Simon Willison — Comment on Feeling sad about AI \(11 Sep 2026\)](#)

Read the complete short comment. Focus on the move from feeling displaced to using existing engineering experience to solve more ambitious problems. This is a practitioner's perspective on adapting to change.

Skip only if you are deeply familiar with this shift and can give concrete examples of how your engineering judgment increases the value of agent-generated work.

R30 Build a tool you wish existed

Estimated study time: about 10 minutes • Q 4.8 / U 4.5 / S 4.70

[Simon Willison — GIF optimization tool using WebAssembly and Gifsicle \(2 Mar 2026\)](#)

Read the full chapter, including its screenshots and short animated demonstration. Follow the idea, result and follow-up decisions. Identify something you would like to build now that an agent can attempt unfamiliar implementation work with your guidance.

Skip only if you are deeply familiar with taking a personal need through an agent-assisted build to a checked result, and can explain where your own expertise shaped the outcome.

R31 Revisit what agents can complete

Estimated study time: under 5 minutes • Q 4.5 / U 4 / S 4.33

[Peter Steinberger — Shipping at Inference-Speed \(28 Dec 2025\)](#)

Read only “What Changed Since May” and “A Concrete Example: VibeTunnel”. Compare the earlier failure with the later reported success and choose a failed task to retry. The excerpt is a historical progress story; use the guide's current model comparison for purchasing decisions. [R26](#) assigns a different section.

Skip only if you are deeply familiar with reassessing agent capabilities and have recently rerun an older failed task with a newer setup, recording what changed and what remained difficult.

Resource R32

Choose how to develop and check code

Estimated study time: about 10 minutes • Q 4.5 / U 4.8 / S 4.61

[Addy Osmani — Agentic Code Review \(15 Jun 2026\)](#)

Read “Everyone is solving a different problem”, “Should we just let AI review more of it?” and “What to actually do” for context on risk-based review. Apply the course policy below to your own change.

Skip only if deeply familiar with choosing code authorship, verification and review depth from the consequences of failure and how reliably defects can be found.

Choose a development workflow

Critical means failure could cause meaningful loss or harm. Easily identifiable means important defects can reliably be found before that harm occurs. Classify the change or component, rather than treating the whole project as one category.

Use the column for criticality and the row for defect visibility. These four choices are the course’s operating policy.

Important defects	Not critical	Critical
Easy to identify	Vibecode and use AI writes the code. No code review.	Vibecode and vibecheck AI writes the code. Use /goal to find defects through executed checks.
Hard to identify	Vibecode then review Iterate with AI until the version is stable, then review the code.	Write by hand and audit with AI You write the implementation. AI hunts for bugs and exploits.

Vibechecking means testing observable behavior against explicit criteria. With /goal, require executed checks and result evidence; its evaluator judges the transcript ([R16](#)). A stable version has settled enough in scope and behavior for a coherent code review.

Why separate implementation and bug finding

For critical code with hard-to-identify defects, combine human implementation with AI vulnerability hunting to seek different failure modes. In audit-heavy projects, this puts AI on the larger workload. Check that auditing dominates cost on your project; different authorship does not guarantee independent mistakes.

When redundancy is feasible, keep one hand-coded implementation and several vibecoded implementations, then compare their behavior against the same requirements.

Apply the matrix to your project

During Day 2’s handoff session, choose a cell and name a hidden failure that could change your choice. Record who implements, what gets checked and when code review occurs. Reassess when the consequences or defect visibility change. For Web3, assess transaction contents as well as the visible interface.